Unit –03
" Arrays and String"

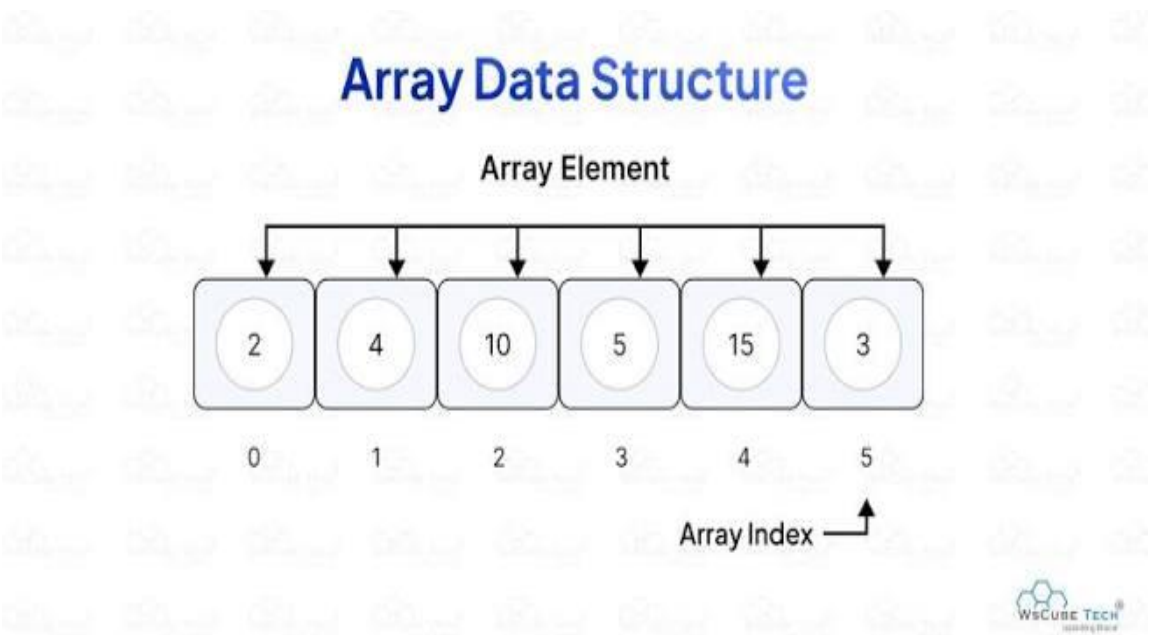
Topics List :-

- Arrays and Its Operations .
- Types of Arrays Including One-Dimensional Arrays and Multi-Dimensional Arrays.
- Strings and String Operations .
- String Comparison and String Manipulation Methods .
- String class.
- String Buffer and String Builder .

Topic –01
" Arrays and Its Operations "

- An **array** is one of the most fundamental data structures in computer science. It is a linear collection of elements, where all elements are of the **same data type** and stored in **contiguous (adjacent) memory locations. It** is one of the most popular and simple data structures used in programming.
- **Syntax of Array Declaration and Initialization :-**

  Arr[] Array Name={elements}



- **Array Operations:-**
  - **Accessing operation :-** Accessing Operation is a type of operation in which the element of Array has been access through index number .

    Example -

    Int[] number={10,20,30,40,50};

    Int first = number[0];

    Int second = number[1];

    Etc.
  - **Insertion operation :-** Insertion Operation is a type of operation in which the element has been inserted into an array using its indexing number and we can also insert element in array in between existing code .

    Example -

    Int[] number = new int[3];

    number[0] = 10;

    number[1] = 20;

    number[2] = 30;

    If we want to insert an element in the middle of the existing data

    Int[] number = {10,20,30,0};

    arr[3] = arr[2];

    arr[2] = arr[1];

    arr[1] = 99;

- **Traversal Operation :-** Traversal operation is a type of operation in which we use for loop for traversing the array we use index number and array.length as well.

  Example-

```
Int[] number = {10,20,30,40,50};
For(int I =0; I<number.length; I++)
{
 System.out.println("Locker" + i + " Contains : " +numbers[i]);
 }
```

- **Deletion Operation :-** Deletion Operation is a another type of operation which is use to delete an particular element from array .

  Example -

```
Int number = {10,20,30,40,50}
Int delete index = 1;
Int current size = 5;
Int current size = array.length;
For(int I =deleteindex; I<currentsize-1 ; I++)
{
  Arr[i] = arr[I+1];
 }
```

```
Arr[current size –1 ] =0;
For(int num: arr )
{
  System.out.println(num + " " );
}
```

- **Updation Operation :** updation operation in array is use when we already define an array and in middle we have to update the particular index value in array .

  For Example :

  ```
  In[] number = {10,20,30,40,50}
  Number[2] = 73;
  ```

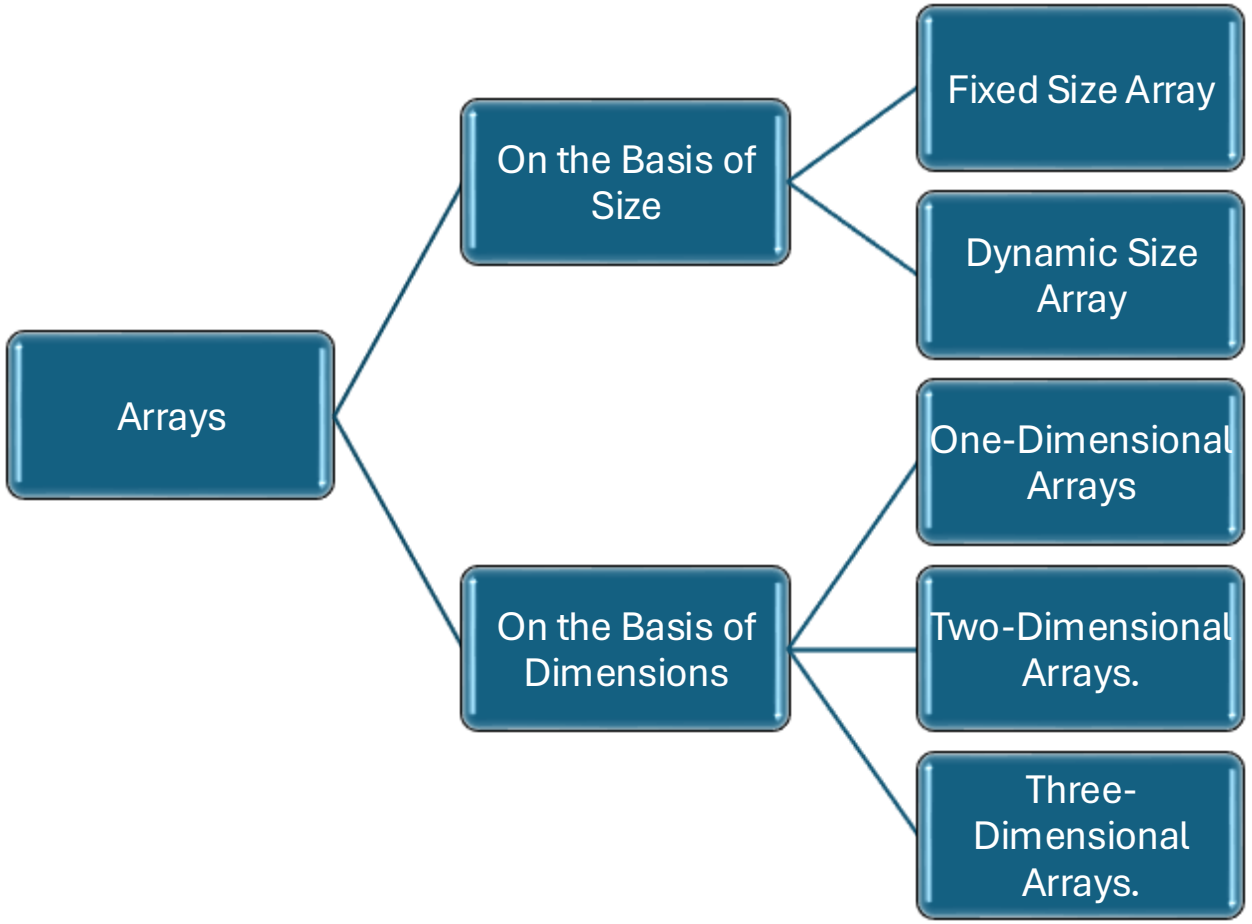
- **Sorting operation :** Sorting Operation in array is use to sort the unsorted array . For this first we have to import library Then using Array. Sort function.

  For Example :

  ```
  Import java.util.Arrays;
  Arrays. Sort(arrayname) ;
  ```

Topic –02
" Types of Arrays Including One-dimensional Arrays and Multi-dimensional Arrays "

- **Types of Arrays :-**



- **On the Basis of Size:-** On the basis of size Arrays are categorized into two types .
- **Fixed Size Arrays –** Fixed Size Array is a Type of Arrays which can't be changed once created or the Size of Array have been inserted. memory has been Allocated only for the size which has been mentioned in square Brackets . Fixed size arrays are not sufficient when the elements are unknown.
- **Syntax –** datatype[] array name = new datatype[size];
- **Dynamic Size Arrays –** Dynamic Size Array is a Type of Arrays Which can Grow and Shrink Automatically during Program Execution . Java Uses ArrayList class (imported from java.util.ArrayList). Unlike Fixed-Size Array , ArrayList Requires you to use Wrapper Classes (Like Integer , Double , String) Instead of Using Primitive data types (int, float , double).
- **Syntax –** ArrayList<wrapperClass>Arrayname = new ArrayList<>();
- **On the Basis of Dimensional:-** On the Basis of dimensional Arrays are Categorized into three types.
- **One-Dimensional Array(1-D Array) : -** One-dimensional array is a type of array which have a single row of elements . Means in this type of array we have the size of array how many elements have been stored .
- **Syntax –**

  Use this Syntax when we know how many stored in particular array but don’t know what are the items yet .

  Datatypes[] array name = new datatypes[size].

  Use this Syntax when we know what are the elements get stored in array .

  Datatypes[] array name = {value 1, value 2 , value 3}.

- **Two-Dimensional Arrays:-** Two-Dimensional array is a type of array which have two dimension its structure is like a matrix .

  Syntax – int[][] matrix2D = new int[][].

  int[][] matrix2D(3 * 3) = {{1,2,3}, {4,5,6}, {7,8,9}}.

- **Three-Dimensional Arrays:-** Three-Dimensional Array is a type of array which have three dimension imagine this as a Cube structure .

  Syntax - int[][][] matrix3D = new int[][][].

  int[3][3][3] matrix3D(3*3*3) =

  {{{1,2,3}, {4,5,6}, {7,8,9}}, {{10,11,12} ,{13,14,15},{16,17,18}}, {{19,20,21}, {22,23,24}, {25,26,27}}};

  Int[2][2][2] matrix3D(2*2*2) = {{{1,2},{3,4}} , {{5,6},{7,8}}} .

Topic :-03
" Strings and Operations "

- Strings is an Object Used to Store Sequence of Characters enclosed in double quotes . It provides methods for handling text data. It is used to hold and manipulate text .
- Strings are Immutable means their Values cannot be changed after creation.

Syntax :- Sting str = new String(" String ");

Examples :- String str = " java " .

String Operations :- Followings are the Operations comes under String Operations :-

- Inspection Operations.
- Comparison Operations.
- Transformation / manipulation Operations.
- Searching Operations.

- **Inspection operation :-** These is a kind of String Operation allows you to check properties of a string or look at individual characters without altering anything .

  $ Length() : Returns the total number of characters.

  $ CharAt(index) : Returns the character at a specific-position (0-indexed).

  $ IsEmpty() : Checks if the String length is 0.

  Examples :

  String str = " Hello ".

  System.out.println(str.length());

  System.out.println(str.charAt(2));

  System.out.println(str.isEmpty());

- **Comparison operation :-** These Operations evaluate two strings against each other and always use methods likes .equals() , never use == to compare the text.

  $ Equals(String another) : Compares characters exactly (Case-sensitive).

  $ EqualsIgnoreCase(String another): Compares Characters While ignoring upper/lowercase differences.

  $ CompareTo(String another): Compares strings alphabetically. Returns 0 if equal, a negative number if first string comes earlier , and a positive number if it comes later.

Examples :

```
String s1 = "Hello";
String s2 = "hello";
System.out.println(s1.equals(s2));
System.out.println(s1.equalsIgnoreCase(s2));
System.out.println("A".compareTo("B"));
```

- **Transformation/manipulation Operation :-** These Operations modify the text layout , they leave the original string untouched and output a new string .

$ SubString(startIndex, endIndex): Extracts a piece of string. It Starts at StartIndex and stop just before endIndex.

$ ToLowerCase() / toUpperCase(): Converts the entire string's casting.

$ Concat(String str): Appends another string to the end (same as using the + Operator).

$ Trim(): Removes blank spaces from the very beginning and very end of the string.

$ replace(older char , new char): Swaps out characters or whole words.

Examples :

```
String = " learn java ".
System.out.println(text.trim());
System.out.println(text.upperCase());
```

- **Searching Operation:-** Searching operation is another type of array operation helps you to locate specific character or sequences inside a larger body of text.

$indexof(Substring): Return the index number where the target text first appears Return –1 if isn't found.

$lastIndexof(substring): Returns the index number where the target text last appears.

$contains(CharSequence s): Returns a simple true and false if the text exists inside.

$startWith(prefix)/endWith(suffix): checks if a string begins or end with specific character.

Examples:

```
String sentence = " Java is fun , Java is powerful "
System.out.println(sentence.Indexof("Java"));
System.out.println(sentence.lastIndexof("Java"));
System.out.println(sentence.contains("fun"));
System.out.println(sentence.endsWith("powerful"));
```

Topic :-04 "String comparison and String Manipulation Methods"

- String Comparison :- String Comparison is a kind of operation which check two strings if they are equal or find out which one comes first.

String comparison includes :

[Note: Never Use == for text comparison]

.equals() :- Case-Sensitive Matching .

Checks if two string have the exact same characters, including matching uppercase and lowercase letters.

Syntax:- boolean

For example:

```
String s1 = "JAVA";
String s2 = "java";
System.out.println(s1.equals(s2));
```

.equalsIgnorecase():- Case-Insensitive Matching

Checks if the text matches, completely ignoring whether the letters are uppercase or lowercase.

Syntax : boolean result = str1.equalsIgnoreCase(str2).

For example:

```
System.out.println(s1.equalsIgnoreCase(str2));
```

CompareTo() - Alphabetical Sorting

Compares two string alphabetically (lexicographically)

Syntax: int result = str1.compareTo(str2);

Return 0 if both string are exactly equal.

Return negative number if str1 comes before str2.

Return Positive number if str1 comer after str2.

For Example:

```
System.out.println("Apple".compareTo("Banana"));
```

- **String Manipulation Methods:-** These Operations allow you to change , clean up , or combine the text strings.

Substring (Extracting a piece of text) : Extracts a smaller portion out of a larger string. It starts cutting at the startIndex and stop right before the end Index.

Syntax: String sub = str.substring(startIndex, endIndex);

Changing Case: Converts all characters in the string into entirely lowercase or uppercase letter.

Syntax: String lower = str.toLowerCase();

#Replacing Characters: Search the string for a specific character or sequence of words and replaces them with a new value.

Syntax: String cleanStr = str.replace(old value, new value).

#Trimming Whitespaces(trim()) : Removes any annoying blank spaces at the very beginning and the very end of a string (it does not affect the spaces in the middle).

Syntax: String clean = str. trim();

#Concatenation (Combining Strings): Joins two strings together end-to-end. You can use the explicit .concat() methods or simply use the " + "Operator shortcut.

Syntax: String combined = str1.concat(str2); or String combined = str1 + str2;

Topic :- 05
" String Class"

In java the String class is a built-in, predefined final class located in java.lang package that represents a sequence of characters. Unlike primitive types like int, char, a string is a non – primitive object.

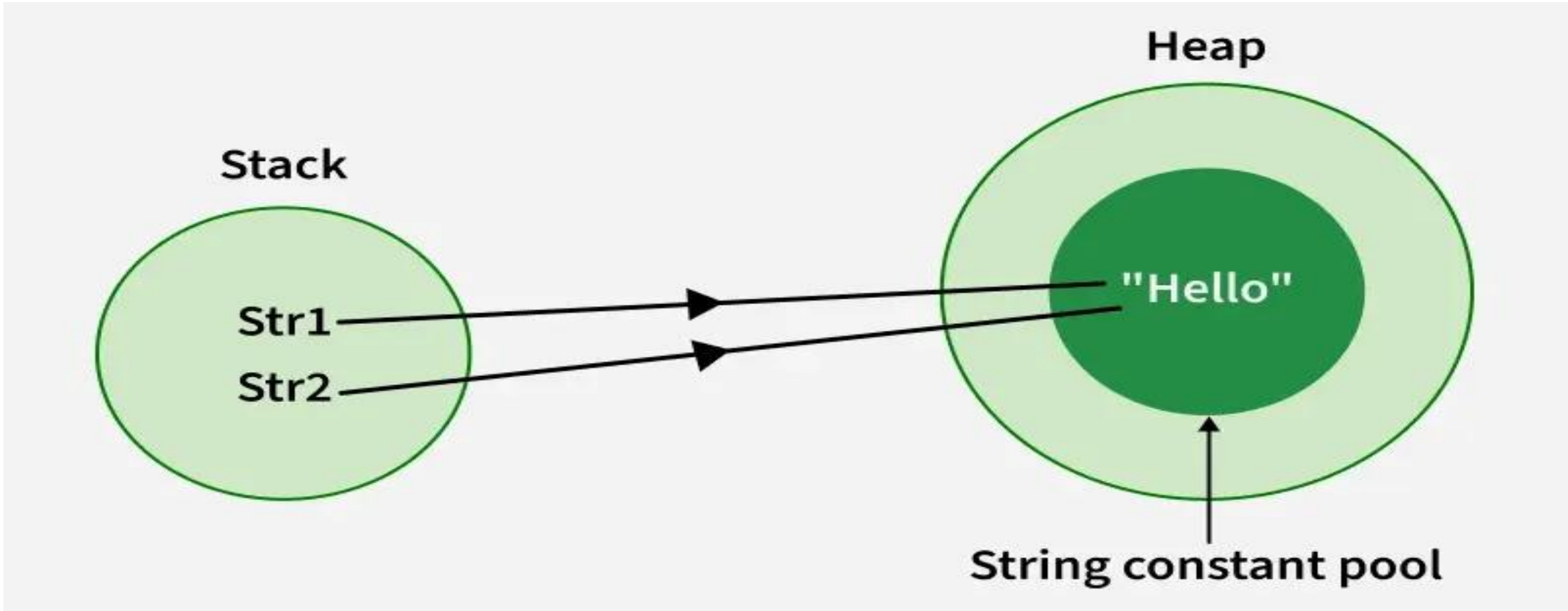
Core characteristics :-

Immutability :- Once a string object is created , its value cannot be change .

Thread Safety :- Because they are immutable, strings can be safely shared across multiple tread without synchronization.

Memory optimization:- Java Optimizes memory usage for strings by utilizing a dedicated area of the head called String Constant pool(SCP).

Because strings are used so frequently, Java optimizes memory usage through a special memory region inside the Heap called the **String Constant Pool (SCP)**.


Heap
Stack
"Hello"
Str1
Str2
String constant pool

The String Class provides rich set of built-in manipulation methods

- Length().
- chatAt(int index).
- Substring (start index , end index).
- To Lowercase().
- Contains(charsequence s).

Topic –06
"String Buffer and String Builder"

`The Standard String` class is **immutable** (unchangeable), modifying a string repeatedly creates a lot of temporary garbage objects in memory.

To solve this problem, Java provides **`String Buffer`** and **`StringBuilder`**. Both of these classes represent **mutable** sequences of characters, meaning you can append, insert, or delete characters without creating new objects in memory.

String Buffer:- String buffer is a class in Java that is used to create **mutable (modifiable)** strings.

This means **you can change the contents of the string without creating a new object.**

Syntax :- StringBuffer sb = new StringBuffer("Hello");

Common Methods of StringBuffer:-

- **append():-** Adds text at the end.

Example:

```
StringBuffer sb = new StringBuffer("Java");
sb.append(" Programming");
System.out.println(sb);
```

- **insert():-** Adds text at a specified position.

Syntax: sb.insert(index, "text");

Example:

```
 StringBuffer sb = new StringBuffer("Java");
sb.insert(4, " Language");
System.out.println(sb);
```

- **replace():**-Replaces part of the string.

Syntax: sb.replace(startIndex, endIndex, "text");

Example:

```
StringBuffer sb = new StringBuffer("Java Language");
sb.replace(5, 13, "Programming");
System.out.println(sb);
```

- **delete():-**Deletes characters.

Syntax: sb.delete(startIndex, endIndex);

Example:

```
StringBuffer sb = new StringBuffer("Java Programming");
sb.delete(4, 16);
System.out.println(sb);
```

- **reverse():-**Reverses the string.

Example:

```
StringBuffer sb = new StringBuffer("Java");
sb.reverse();
System.out.println(sb);
```

- **length():-**Returns the length.

Example:

```
StringBuffer sb = new StringBuffer("Java");
System.out.println(sb.length());
```

- **String builder:- string builder** is also a class used to create **mutable string** .It works almost exactly like String buffer , but it is **faster** because it is **not thread-safe (not synchronized)**.means when two thread run exactly same time , their operation can overlap leading to unexpected or incorrect result . While in the string buffer once thread complete their operation and get lock then only next thread will run .

  Common methods used in String Builder :-

  i)Append().

  ii)insert().

  iii)replace().

  iv)delete().

  v)reverse().

  vi)length().

All methods are same there is only one difference that we have to use the String builder at the place of string buffer.